SIMATS ENGINEERING

Department of Computer Science and Engineering ASSESSMENT TOOL2 - CONCEPT MAPPING

Course Code:	CSA1205	**Course Title:**	Computer Architecture for Multicore Systems
CO Assessed:	**CO1**	**Assessment:**	**ASSESSMENT TOOL 2-** CONCEPT MAPPING
Weightage:	**35%**	**Total Marks:**	**25**
CO1 Statement:	Analsze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	**GAYATHERI S**	**Reg.No.:192521306**	
Department:	IT	**Date:15/07/2026**	

Instruction to Students

1. Answer two questions as per the Instruction.
2. Each question carries 12.5 mark.
3. Only the appropriate Tools can be used
4. Time allotted for the exam is 60 min

Code of Conduct:

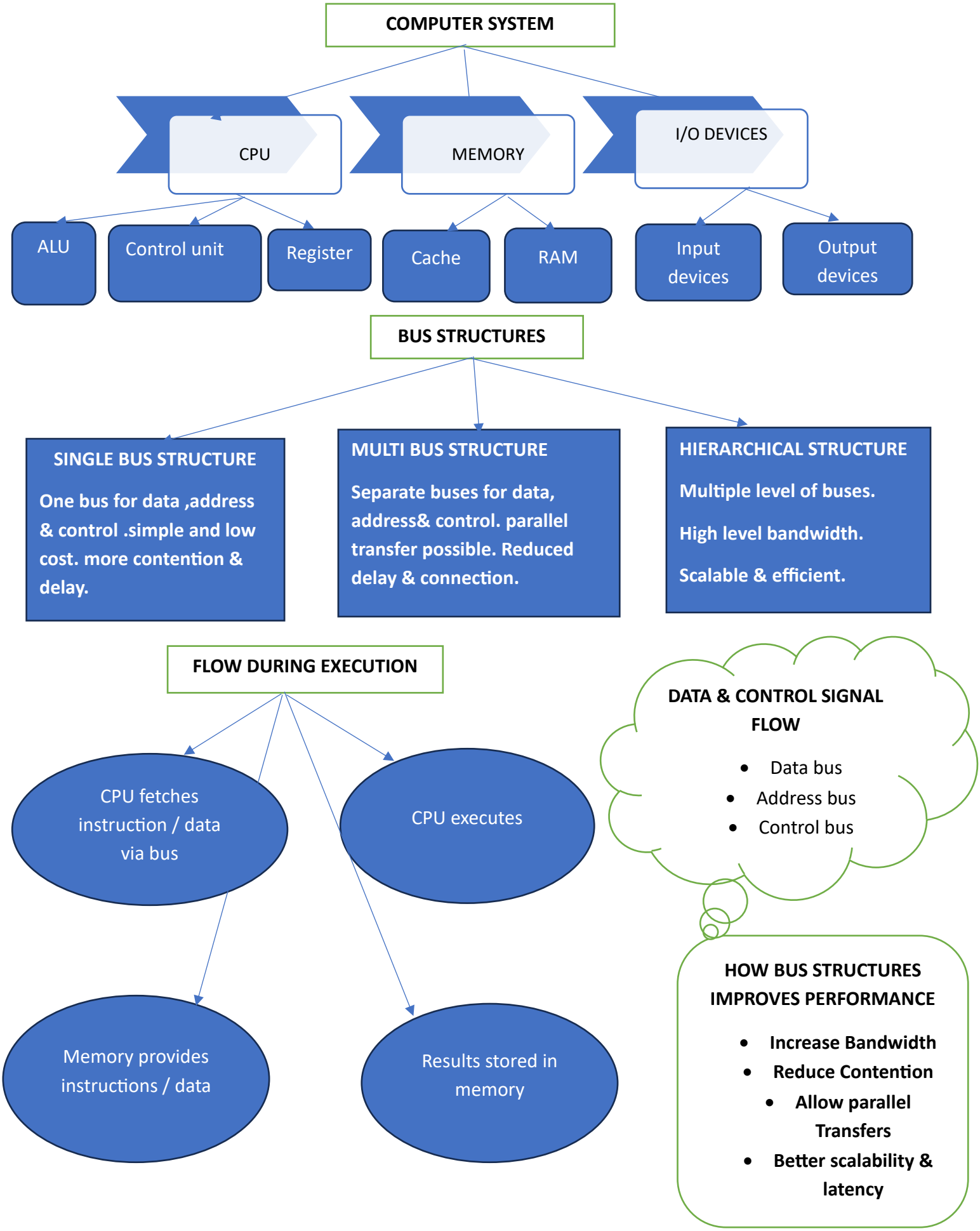
I GAYATHERI S (192521306) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

GAYATHERI S

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.
4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.


COMPUTER SYSTEM
CPU
MEMORY
I/O DEVICES
ALU
Control unit
Register
Cache
RAM
Input devices
Output devices
BUS STRUCTURES
SINGLE BUS STRUCTURE
One bus for data ,address & control .simple and low cost. more contention & delay.
MULTI BUS STRUCTURE
Separate buses for data, address& control. parallel transfer possible. Reduced delay & connection.
HIERARCHICAL STRUCTURE
Multiple level of buses.
High level bandwidth.
Scalable & efficient.
FLOW DURING EXECUTION
CPU fetches instruction / data via bus
CPU executes
Memory provides instructions / data
Results stored in memory
DATA & CONTROL SIGNAL FLOW
Data bus
Address bus
Control bus
HOW BUS STRUCTURES IMPROVES PERFORMANCE
Increase Bandwidth
Reduce Contention
Allow parallel Transfers
Better scalability & latency
POSSIBLE DELAYS / BOTTLENECKS
1.Bus Contention
2.Limited Bus Bandwidth
3.Slow Memory Access
4.I/O Device Latency

INSTRUCTION & EXECUTION CYCLE

1.FETCH
fetch information from memory and update PC

2.DECODE
decode instruction and determine operation

3.EXECUTE
perfoem operation and access memory

CPU PERFORMANCE PARAMETERS

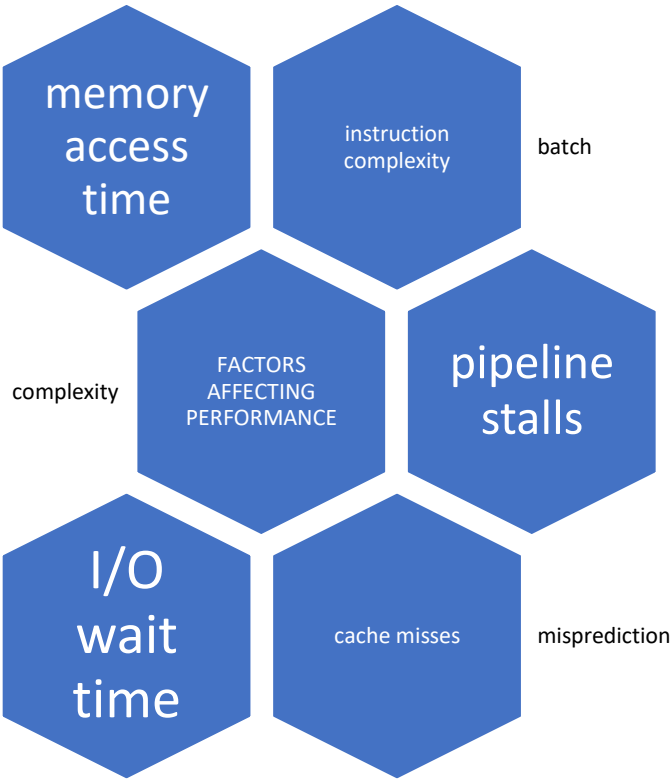
CLOCK CYCLES
- Number of cycles for execution

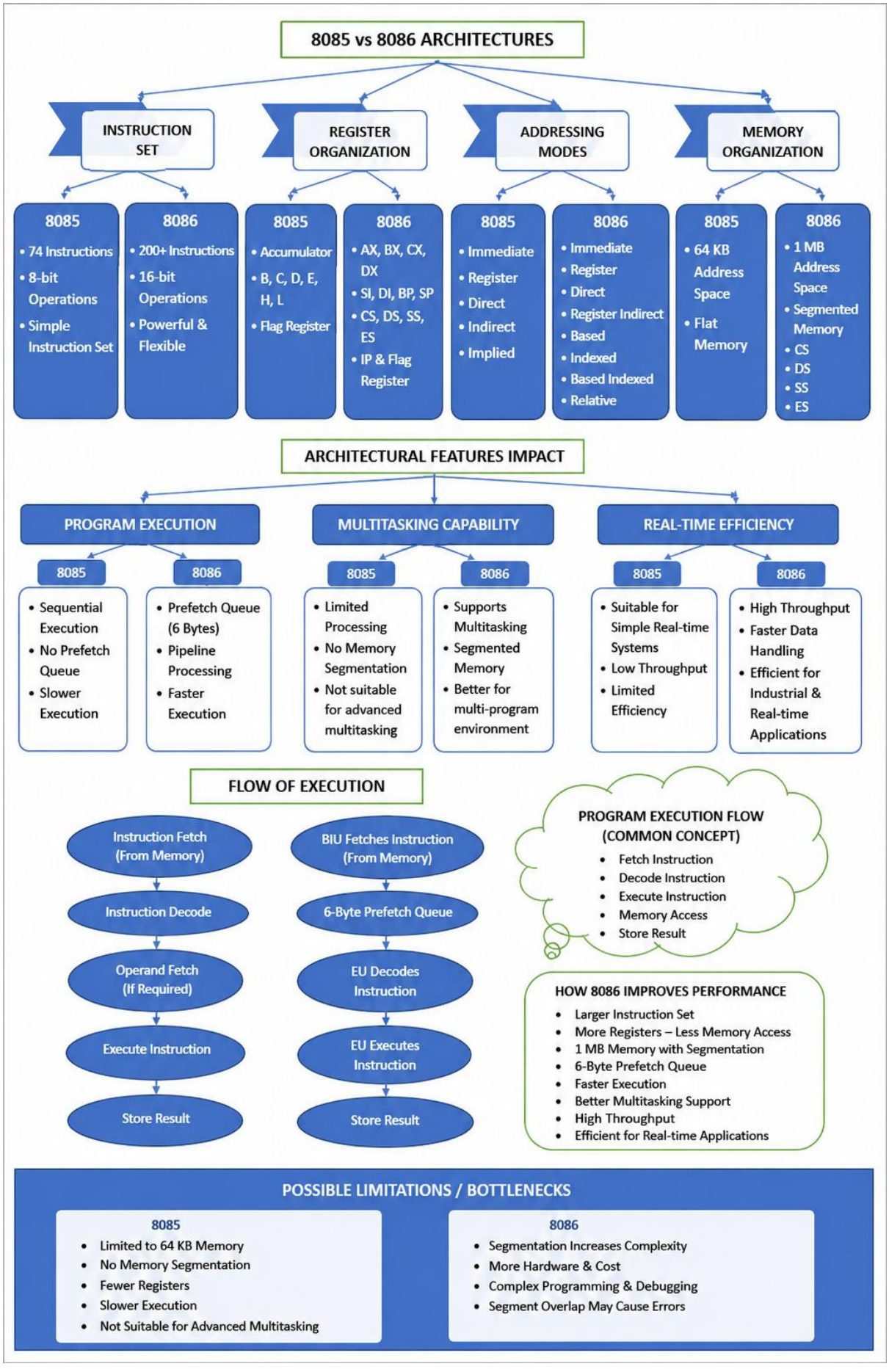
CPI
- total clock cycles/total instructions

EXECUTION TIME
- CPI clock cycles X clock cycle time

BENCHMARKING
- Run standard benchmarks program
- Measure performance parameters
- Compare system /configurations

USED TO EVALUATE
- **CPU efficiency**
- **System performance**
- **Design improvements**




8085 vs 8086 ARCHITECTURES
INSTRUCTION SET
REGISTER ORGANIZATION
ADDRESSING MODES
MEMORY ORGANIZATION
8085
• 74 Instructions
• 8-bit Operations
• Simple Instruction Set
8086
• 200+ Instructions
• 16-bit Operations
• Powerful & Flexible
8085
• Accumulator
• B, C, D, E, H, L
• Flag Register
8086
• AX, BX, CX, DX
• SI, DI, BP, SP
• CS, DS, SS, ES
• IP & Flag Register
8085
• Immediate
• Register
• Direct
• Indirect
• Implied
8086
• Immediate
• Register
• Direct
• Register Indirect
• Based
• Indexed
• Based Indexed
• Relative
8085
• 64 KB Address Space
• Flat Memory
8086
• 1 MB Address Space
• Segmented Memory
• CS
• DS
• SS
• ES
ARCHITECTURAL FEATURES IMPACT
PROGRAM EXECUTION
MULTITASKING CAPABILITY
REAL-TIME EFFICIENCY
8085
• Sequential Execution
• No Prefetch Queue
• Slower Execution
8086
• Prefetch Queue (6 Bytes)
• Pipeline Processing
• Faster Execution
8085
• Limited Processing
• No Memory Segmentation
• Not suitable for advanced multitasking
8086
• Supports Multitasking
• Segmented Memory
• Better for multi-program environment
8085
• Suitable for Simple Real-time Systems
• Low Throughput
• Limited Efficiency
8086
• High Throughput
• Faster Data Handling
• Efficient for Industrial & Real-time Applications
FLOW OF EXECUTION
Instruction Fetch (From Memory)
Instruction Decode
Operand Fetch (If Required)
Execute Instruction
Store Result
BIU Fetches Instruction (From Memory)
6-Byte Prefetch Queue
EU Decodes Instruction
EU Executes Instruction
Store Result
PROGRAM EXECUTION FLOW (COMMON CONCEPT)
• Fetch Instruction
• Decode Instruction
• Execute Instruction
• Memory Access
• Store Result
HOW 8086 IMPROVES PERFORMANCE
• Larger Instruction Set
• More Registers – Less Memory Access
• 1 MB Memory with Segmentation
• 6-Byte Prefetch Queue
• Faster Execution
• Better Multitasking Support
• High Throughput
• Efficient for Real-time Applications
POSSIBLE LIMITATIONS / BOTTLENECKS
8085
• Limited to 64 KB Memory
• No Memory Segmentation
• Fewer Registers
• Slower Execution
• Not Suitable for Advanced Multitasking
8086
• Segmentation Increases Complexity
• More Hardware & Cost
• Complex Programming & Debugging
• Segment Overlap May Cause Errors

ADDRESSING MODES AND INSTRUCTION TYPES THEIR IMPACT AND REAL-WORLD USE

IMMEDIATE ADDRESSING

- Operand is part of the instruction
- Data is given immediately
- No memory address needed

DIRECT ADDRESSING

- Address field contains the memory address of operand
- Operand is in memory location

REGISTER ADDRESSING

- Operand is in a CPU register
- Fast access
- No memory access

INDIRECT ADDRESSING

- Address field contains the address of memory location
- That location contains the actual operand address

INDEXED ADDRESSING

- Effective address = Index register + Displacement
- Useful for arrays and tables

INSTRUCTION TYPES

DATA TRANSFER INSTRUCTIONS

- MOV, MVI, LDA, STA, LXI
- Move data between registers, memory and I/O
- All addressing modes can be used

ARITHMETIC INSTRUCTIONS

- ADD, SUB, ADC, SBB, INC, DEC
- Perform calculations on data
- Use all addressing modes

CONTROL INSTRUCTIONS

- JMP, JZ, JNZ, CALL, RET
- Change the flow of execution
- Usually use Direct, Indirect or Indexed addressing

EFFECT OF ADDRESSING MODES

ADDRESSING MODE	MEMORY ACCESS TIME	INSTRUCTION SIZE	EXECUTION SPEED	REMARKS
IMMEDIATE	No memory access (Fastest)	Larger (contains data)	High	Best for constants and small data
DIRECT	One memory access (Slower than register)	Medium (address field)	Medium	Good for global variables
REGISTER	No memory access (Fastest)	Small (register number)	Very High (Fastest)	Best for temporary data
INDIRECT	Two memory accesses (Slowest)	Medium	Low (Slowest)	Used for pointers and dynamic data
INDEXED	One memory access + index calculation	Medium to Large (index + displacement)	Medium to High	Best for arrays, tables, buffers

REAL-WORLD SCENARIOS

IMMEDIATE ADDRESSING

- Used in initializing variables
- Setting constants
- Examples: MOV A, #55H
- Microcontroller configuration

DIRECT ADDRESSING

- Accessing global variables
- I/O mapped devices
- Examples: LDA 2000H
- Used in embedded systems

REGISTER ADDRESSING

- Fast calculations
- Data manipulation inside CPU
- Examples: ADD R1, R2
- Used in high-speed applications

INDIRECT ADDRESSING

- Used in linked lists
- Pointers and dynamic memory
- Examples: MOV A, @R0
- Used in operating systems

INDEXED ADDRESSING

- Array and table processing
- Database navigation
- Examples: MOV A, ARRAY[SI]
- Used in data structures

HOW IT AFFECTS PROGRAM PERFORMANCE

Choice of Addressing Mode → Affects Memory Access Time → Changes Instruction Size → Influences Execution Speed → Affects Overall Program Efficiency

KEY TAKEAWAYS

- Register addressing is the fastest due to no memory access.
- Immediate addressing is best for constant values.
- Direct addressing is simple and widely used for global data.
- Indirect addressing adds flexibility but increases time.
- Indexed addressing is ideal for arrays and structured data.
- Proper addressing mode selection improves execution speed and memory efficiency.

NUMBER SYSTEMS, INSTRUCTION REPRESENTATION, MEMORY STORAGE AND DEBUGGING PROCESS

NUMBER SYSTEMS

BINARY

- Base-2 system
- Uses 0 and 1
- Directly used by CPU
- Example: 101101_2

HEXADECIMAL

- Base-16 system
- Uses 0–9 and A –F
- Compact form of binary
- Example: $2D3F_{16}$

DATA CONVERSION BETWEEN FORMATS

Decimal

↓

Binary (Divide by 2)

↓

Group Binary (4 bits)

↓

Hexadecimal (Convert each group)

↓

Example: $(45)_{10} \rightarrow 0010\ 1101_2 \rightarrow 2D_{16}$

INSTRUCTION REPRESENTATION

- Instructions are first written in Assembly (Mnemonics)
- Assembler converts mnemonics to machine code (Binary)
- Machine code can be represented in Binary or Hexadecimal
- Example:
  MOV A, B
  → 0101 0110 → 56_{16}

MEMORY STORAGE

- Memory stores data and instructions in binary form
- Each memory location has an address
- Hexadecimal is used to represent both address and data
- Example:
  Address: $2A1F_{16}$
  Data : $3C_{16}$

DEBUGGING PROCESSES

- Debugger reads memory content in hexadecimal
- Easy to read and understand
- Used to analyze instructions, registers, memory dumps
- Helps in finding errors and fixing bugs

HOW CPU INTERPRETS AND EXECUTES INSTRUCTIONS

Fetch Instruction from Memory (using address) → Read Binary Instruction from Memory (Location) → Decode Instruction (Opcode + Operands) → Execute Instruction (ALU / Control Unit) → Store Result back to Register / Memory

CPU works on Binary. Hexadecimal is used by humans for easy representation and understanding.

BINARY vs HEXADECIMAL REPRESENTATION

ASPECT	BINARY	HEXADECIMAL	EXAMPLE COMPARISON
Base	2	16	Binary: 1101 1010 Hex : DA
Digits Used	0, 1	0–9, A–F	Hex digits represent 4 bits each
Representation Size	Large (more bits)	Compact (fewer digits)	32-bit binary = 8 hex digits
Readability	Difficult for humans	Easy and shorthand	0x1A3F is easier than 0001 1010 0011 1111
Usage	Used by CPU	Used by Programmers, Debuggers	Memory addresses, Machine code, Registers

WHY HEXADECIMAL IS COMMONLY USED

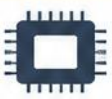
COMPACT REPRESENTATION

- 4 binary bits = 1 hex digit
- Reduces code size and increases readability

EASY TO READ & WRITE

- Shorter and manageable
- Less chance of human error

STANDARD IN SYSTEM DESIGN



- Memory addresses, opcodes, register values in hex
- Used in datasheets, manuals, and tools

EFFICIENT DEBUGGING & ANALYSIS

- Memory dumps and registers shown in hex
- Easier to trace and compare data

INDUSTRY PRACTICE

- Assemblers, linkers, loaders use hex
- Widely used in embedded systems and OS development

DEBUGGING WORKFLOW USING HEXADECIMAL

Program Runs → Set Breakpoint → CPU Stops at Instruction → View Registers & Memory (in Hex) → Analyze Data & Trace Program → Find Bug & Fix Code

KEY TAKEAWAYS

- Binary is the language of the CPU; hexadecimal is the language of humans.
- Instructions are stored in memory in binary but represented in hexadecimal for convenience.
- Hexadecimal simplifies memory addresses, data representation, and debugging.
- Understanding number systems and their conversion is essential for system design and low-level programming.